

Multimodal Large Model + Robotic Arm Handling

Function Description

After the program runs, input the color block handling command through the terminal. The large model will plan the actions to complete the color block handling task, and then the program will control the robotic arm to handle the color blocks in order.

Startup

Open the terminal and enter the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

Then input the color block handling command in the terminal. You can refer to the following examples:

```
Place the green block on the desktop above the red block
```

The available positions are: front, back, left, right, top - these five positions.

```
root@yahboom:/# ros2 run text_chat text_chat
user input: Place the green block on the desktop above the red block
okay@, let me think for a moment... \[INFO] [1775123877.111506338] [text_chat_node]: Decision making AI planning:1. Call 'seewhat()' to observe the environment and check the positions of t
he green block and the red block.
2. Call 'grasp_obj(x1, y1, x2, y2)' to grasp the green block, where '(x1, y1, x2, y2)' are the outer frame coordinates of the green block.
3. Call 'change_pose(x1, y1, x2, y2, "green", "red", 5, tar_size)' to place the green block on top of the red block, where '(x1, y1, x2, y2)' are the outer frame coordinates of the red block
, 'tar_size' indicates the height of the red block.
[INFO] [1775123880.813478479] [text_chat_node]: "action": ['seewhat()'], "response": Okay, I'm going to place the green block on top of the red block. Let me first check where they are!
[INFO] [1775123889.251847512] [text_chat_node]: "action": ['grasp_obj(350, 220, 510, 380)'], "response": I see the red block and the green block now. Let me grab the green one first!
[INFO] [1775123921.838441017] [text_chat_node]: "action": ['change_pose(30, 200, 170, 340, 'green', 'red', 5, 5)'], "response": Got the green block! Now I'll place it right on top of the red
block.
[INFO] [1775123944.254675769] [text_chat_node]: "action": ['finishtask()'], "response": Done! The green block is now perfectly placed on top of the red block. Everything's in order!
```

The program will first take a photo to get the positions of the green block and the red block; then it will control the robotic arm to grip the green block and place it above the red block.

Task Planning

1. Call `seewhat()` to observe the environment and check the positions of the green block and the red block.
2. Call `grasp_obj(x1, y1, x2, y2)` to grasp the green block, where `(x1, y1, x2, y2)` are the outer frame coordinates of the green block.
3. Call `change_pose(x1, y1, x2, y2, "green", "red", 5, tar_size)` to place the green block on top of the red block, where `(x1, y1, x2, y2)` are the outer frame coordinates of the red block, `tar_size` indicates the height of the red block. The default is 3 centimeters, unless otherwise specified.

Core Code Analysis

change_pose Function

Source code path: `LargeModel_ws/src/largemodel1/largemodel1/action_service.py`

```
def change_pose(self, x1, y1, x2, y2, src, tar, side, tar_size):
    """
    Execute pose change operation
    Parameter description:
        x1, y1, x2, y2: Top-left and bottom-right corner coordinates of the outer frame of
        the gripped color block
        For example, in the above case, these are the top-left and bottom-right corner
        coordinates of the outer frame of the green block
        src: Color of the gripped color block
        For example, in the above case, this is the green block
        tar: Color of the placed color block
        For example, in the above case, this is the red block
        side: Placement position
        Front, back, left, right, top correspond to values 1, 2, 3, 4, 5 respectively
        For example, in the above case, this is 5, indicating top
        tar_size: Target size parameter
    """

    # Process source object name (gripped color block color)
    src_color = src.strip("`\"") # Remove single and double quotes
    self.cur_down_name = src_color
    self.cur_down_pose[self.cur_down_name] = []

    # Start pose change node
    cmd1 = "ros2 run largemodel_arm Change_Pose"
    subprocess.Popen([
        "gnome-terminal",
        "--title=Change_Pose",
        "--",
        "bash",
        "-c",
        f"{cmd1}; exec bash",
    ])
    time.sleep(8.0)

    # Type conversion
    x1 = int(x1)
    y1 = int(y1)
    x2 = int(x2)
    y2 = int(y2)
    side = int(side)
    tar_size = int(tar_size)

    # Publish target position information
    while not self.object_position_pub.get_subscription_count():
        self.get_logger().info("-----Publishing-----")
```

```

        self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2, y2, side,
tar_size]))
        time.sleep(0.1)

    self.get_logger().info("-----Int16Array Publishing Complete-----
----")
    self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2, y2, side, tar_size]))

    # wait for pose change to complete
    while not self.change_pose_future.done():
        if self.interrupt_flag: # Check interrupt flag
            self.check_close_change_pose()
            # Return robotic arm to home position
            Arm.Arm_serial_servo_write6(90, 120, 0, 0, 90, 30, 1000)
            return
        time.sleep(0.1)

    # Process result
    result = self.change_pose_future.result()
    self.check_close_change_pose()

    if not self.interrupt_flag:
        if result.data == "change_pose_done":
            self.action_status_pub("change_pose_done", x1=x1, y1=y1, x2=x2, y2=y2)
        else:
            self.action_status_pub("change_pose_failed", x1=x1, y1=y1, x2=x2, y2=y2)

    # Reset Future object
    self.change_pose_future = Future()

    # If interrupted, return robotic arm to home position
    if self.interrupt_flag:
        time.sleep(0.5)
        Arm.Arm_serial_servo_write6(90, 120, 0, 0, 90, 30, 1000)

```

Advanced Commands

We can have the robotic arm complete a more complex task: remember the current position of the color block, then place the color block on top of another color block, and then place this block back to its original position. In the text_chat terminal, you can input the following:

Remember the current position of the red square, then place the red square on top of the yellow square, and finally return the red square to its original position.

```

root@yanboom:~/# ros2 run text_chat text_chat
user input: Remember the current position of the yellow square, then place the yellow square on top of the red square, and finally return the yellow square to its original position.
okay@, let me think for a moment... -[INFO] [1775125838.771643428] [text_chat_node]: Decision making AI planning:Follow these steps and output only one action function each time:
1. Call 'seewhat()' to observe the environment;
2. Call the 'compute_pose(x1, y1, x2, y2)' function to record the current position of the yellow block;
3. Call 'grasp_obj(x1, y1, x2, y2)' to grasp the yellow block, where x1, y1, x2, y2 are the outer frame coordinates of the yellow block;
4. Call 'seewhat()' to observe the environment and check the position of the red block;
5. Call the 'change_pose(x1, y1, x2, y2, "yellow", "red", side, tar_size)' function to place the yellow block on top of the red block, where x1, y1, x2, y2 are the outer frame coordinates of the red block, side indicates the placement position, and tar_size indicates the height of the red block;
6. Call 'grasp_from_down_list("yellow")' to find the current position of the yellow block in the placement list;
7. Call the 'return_to_orin("color")' function, where "color" is the color of the target block to be reset, valid values are "red", "blue", "green", or "yellow".
[INFO] [1775125840.892554457] [text_chat_node]: "action": ['seewhat()'], "response": Alright, I'm going to start by checking the environment to locate the yellow block. Let's see what's around!
[INFO] [1775125850.188500235] [text_chat_node]: "action": ["compute_pose(30, 200, 170, 350, 'yellow')"], "response": I've spotted the yellow block! Now, I'll record its current position so I can come back to it later.
[INFO] [1775125867.426430193] [text_chat_node]: "action": ['grasp_obj(30, 200, 170, 350)'], "response": Got it! The yellow block's position is saved. Now, let me grab it and get ready to place it on top of the red block.
[INFO] [1775125900.391096658] [text_chat_node]: "action": ['seewhat()'], "response": I've got the yellow block in hand! Now, let me check where the red block is so I can place it on top.
[INFO] [1775125910.038080689] [text_chat_node]: "action": ["change_pose(300, 200, 480, 370, 'yellow', 'red', 5, 5)"], "response": I see the red block clearly now. Let me place the yellow block right on top of it!
[INFO] [1775125932.023013358] [text_chat_node]: "action": ["grasp_from_down_list('yellow')"], "response": The yellow block is now sitting neatly on top of the red block! Now, let me find its current position in the placement list so I can return it to where it started.
[INFO] [1775125942.352079121] [text_chat_node]: "action": ['return_to_orin('yellow')'], "response": I've found the yellow block in the placement list. Now, I'll return it to its original position. Let's go!
[INFO] [1775125954.040343519] [text_chat_node]: "action": ['finishtask()'], "response": All done! The yellow block is back in its original spot. I've completed every step smoothly. Let me know if you need anything else!

```

Task Planning

1. Call `seewhat()` to observe the environment;
2. Call the `compute_pose(x1, y1, x2, y2)` function to record the current position of the yellow block;
3. Call `grasp_obj(x1, y1, x2, y2)` to grasp the yellow block, where x1, y1, x2, y2 are the outer frame coordinates of the yellow block;
4. Call `seewhat()` to observe the environment and check the position of the red block;
5. Call the `change_pose(x1, y1, x2, y2, "yellow", "red", side, tar_size)` function to place the yellow block on top of the red block, where x1, y1, x2, y2 are the outer frame coordinates of the red block, side indicates the placement position, and tar_size indicates the height of the red block;
6. Call `grasp_from_down_list("yellow")` to find the current position of the yellow block in the placement list;
7. Call the `return_to_orin("color")` function, where "color" is the color of the target block to be reset, valid values are "red", "blue", "green", or "yellow".

Core Code Analysis

The change_pose has been analyzed in previous tutorials. Let's mainly analyze the compute_pose, grasp_from_down_list, and return_to_orin three functions.

compute_pose Record Position Function

Source code located at: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```

def compute_pose(self, x1, y1, x2, y2, name):

    cur_name = name.strip('"\'') # Remove single and double quotes
    self.cur_name = cur_name
    self.cur_pose[self.cur_name] = []
    #self.check_close_compute_pose()
    #Start record position information program
    cmd1 = "ros2 run largemodel_arm Record_pose"
    # cmd3 = "ros2 run --prefix 'gdb -ex run --args' M3Pro_KCF ALM_KCF_Tracker_Node"
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=compute_pose",
            "--",
            "bash",
            "-c",

```

```

        f"{cmd1}; exec bash",
    ]
)
time.sleep(5.0)
#Publish outer frame coordinate information topic data, which will be subscribed to by
the Record_pose node
x1 = int(x1)
y1 = int(y1)
x2 = int(x2)
y2 = int(y2)
self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2, y2]))

```

The Record_pose record position information program source code located at:

`LargeModel_ws/src/largemodel_arm/largemodel_arm/Record_pose.py`

```

#Create subscriber to subscribe to object outer frame information
self.xy_subscription =
self.create_subscription(Int16MultiArray, 'corner_xy', self.GetXYCallback, qos_profile=1)
#Create publisher to publish current position topic message
self.current_pose_position_pub = self.create_publisher(Float32MultiArray, "current_pose", 1)

#Callback function GetXYCallback to process received messages and calculate the object
center point coordinates
def GetXYCallback(self, msg):
    print("msg: ", msg.data)
    print(msg.data[0])
    print(msg.data[1])
    print(msg.data[2])
    print(msg.data[3])
    self.cx = (msg.data[0] + msg.data[2])/2
    self.cy = (msg.data[1] + msg.data[3])/2

#In the TagDetect callback function,
if self.cx!=0 and self.cy!=0:
    cx = self.cx
    cy = self.cy
    #Get the depth value of the object center point
    cz = depth_image_info[int(cy),int(cx)]/1000
    if cz!=0 and self.pub_pose == True:
        #Calculate the current center point position
        pose = self.compute_heigh(cx, cy, cz)
        print("pose: ", pose)
        #Publish object current position topic message
        self.current_pose_position_pub.publish(Float32MultiArray(data=[pose[0], pose[1],
pose[2]]))
        self.pub_pose = False
    #Feedback to the large model that recording the current position is complete
    self.largemodel_arm_done_pub.publish(String(data="compute_pose_done"))

```

grasp_from_down_list Grip Object from Placement List Function

Source code located at: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def grasp_from_down_list(self,color):
    tar_color = color.strip("\")
    #According to the key value, find the target pose value of the robotic arm in the
    placement dictionary
    tar_joints = self.cur_down_pose.get(tar_color)
    #Control the robotic arm to move to the target pose
    Arm.Arm_serial_servo_write6(tar_joints[0], tar_joints[1], tar_joints[2], tar_joints[3],
    tar_joints[4], 0,2000)
    time.sleep(2.0)
    Arm.Arm_serial_servo_write(6, 140, 1000)
    time.sleep(2.0)
    Arm.Arm_serial_servo_write6(90,120,10,10,90,140,2000)
    time.sleep(2.0)
    #Feedback to the large model that gripping the color block from the placement list is
    complete
    self.action_status_pub("grasp_from_down_list_done")
```

return_to_orin Color Block Reset Function

Source code located at: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def return_to_orin(self,name):
    #Get the current color parameter
    tar_name = name.strip("\")
    self.get_logger().info(f"tar_name: {tar_name}")
    self.return_flag = True
    self.get_logger().info('Start to put it to the orin position.')
    self.get_current_end_pos()
    while not self.client.wait_for_service(timeout_sec=1.0):
        self.get_logger().info('Service not available, waiting...')
    #If the current color block color can be found in the dictionary storing the current
    position, then call the inverse kinematics service, passing the corresponding position of
    the color block as a parameter to the service, get the servo angle values of the robotic
    arm, and then control the robotic arm movement.
    if tar_name in self.cur_pose:
        self.get_logger().info('Get the orin position.')
        request = Kinemarics.Request()
        request.tar_x = self.cur_pose[tar_name][0]
        request.tar_y = self.cur_pose[tar_name][1] + 0.01
        request.tar_z = self.cur_pose[tar_name][2] + 0.01
        request.kin_name = "ik"
        request.roll = -1.0
        #request.pitch = self.CurEndPos[4]
        #request.yaw = self.CurEndPos[5]
        self.get_logger().info(f"request: {request}")
        future = self.client.call_async(request)
        future.add_done_callback(self.get_ik_response_callback)
```

